

Practical 2

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn import metrics
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay, precision_recall_fscore_support
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.utils import resample
```

```
In [2]: df = pd.read_csv("../data/emails.csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	infrastructure	
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	0	0		0
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	0	0		0
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	0	0		0
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	0	0		0
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	0	0		0

5 rows × 3002 columns



```
In [4]: df.shape
```

```
Out[4]: (5172, 3002)
```

```
In [5]: df.describe().T
```

```
Out[5]:
```

	count	mean	std	min	25%	50%	75%	max
the	5172.0	6.640565	11.745009	0.0	0.0	3.0	8.0	210.0
to	5172.0	6.188128	9.534576	0.0	1.0	3.0	7.0	132.0
ect	5172.0	5.143852	14.101142	1.0	1.0	1.0	4.0	344.0
and	5172.0	3.075599	6.045970	0.0	0.0	1.0	3.0	89.0
for	5172.0	3.124710	4.680522	0.0	1.0	2.0	4.0	47.0
...	...	...	...	...	...	...	...	...
military	5172.0	0.006574	0.138908	0.0	0.0	0.0	0.0	4.0
allowing	5172.0	0.004060	0.072145	0.0	0.0	0.0	0.0	3.0
ff	5172.0	0.914733	2.780203	0.0	0.0	0.0	1.0	114.0
dry	5172.0	0.006961	0.098086	0.0	0.0	0.0	0.0	4.0
Prediction	5172.0	0.290023	0.453817	0.0	0.0	0.0	1.0	1.0

3001 rows × 8 columns

Without upsampling

```
In [6]: df = df.drop("Email No.", axis=1)
```

```
In [7]: df.isna().sum()
```

```
Out[7]: the          0
to          0
ect         0
and         0
for         0
..
military    0
allowing    0
ff          0
dry         0
Prediction  0
Length: 3001, dtype: int64
```

```
In [8]: sns.distplot(x=df["Prediction"])
plt.show()
```

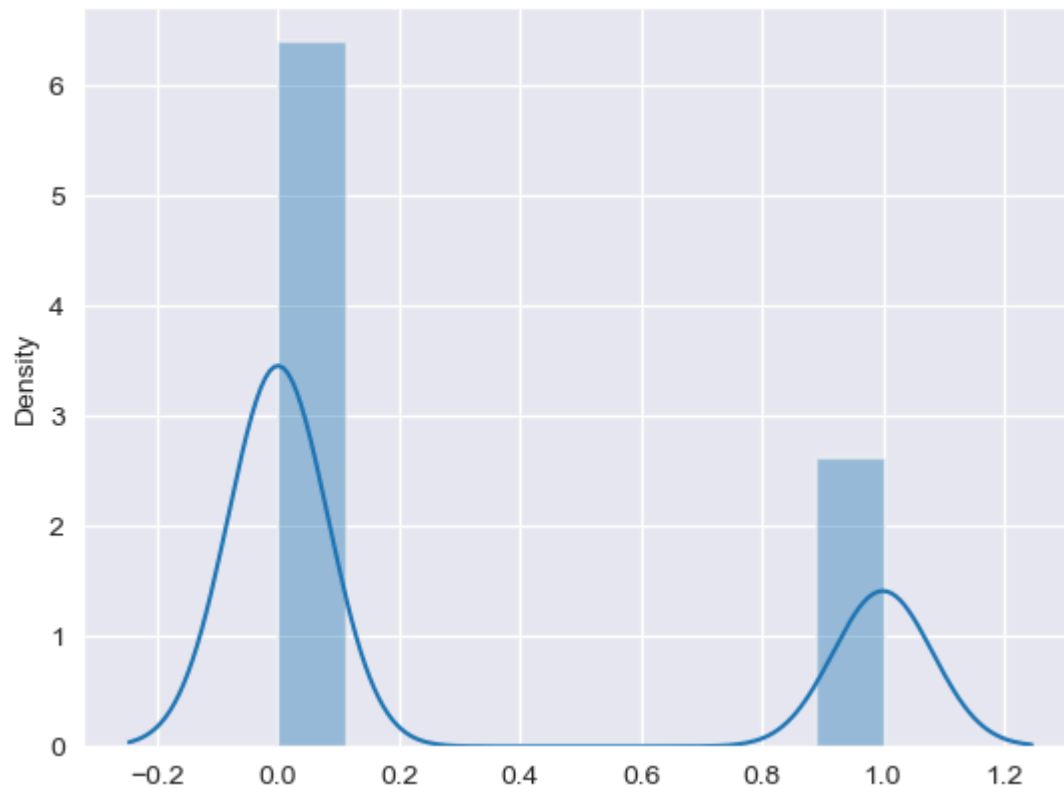
C:\Users\ansha\AppData\Local\Temp\ipykernel_21284\3747884715.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=df["Prediction"])
```



```
In [9]: x = df.drop("Prediction", axis=1)
        y = df[["Prediction"]]
```

```
In [10]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Using KNN

```
In [11]: knn_model = KNeighborsClassifier(n_neighbors=2)
        knn_model.fit(x_train, y_train.values.ravel())
        y_pred = knn_model.predict(x_test)
        print(metrics.classification_report(y_test.values.ravel(), y_pred))
```

	precision	recall	f1-score	support
0	0.90	0.92	0.91	746
1	0.78	0.72	0.75	289
accuracy			0.86	1035
macro avg	0.84	0.82	0.83	1035
weighted avg	0.86	0.86	0.86	1035

```
In [12]: knn_model.score(x_train,y_train.values.ravel())
        knn_model.score(x_test,y_test.values.ravel())
```

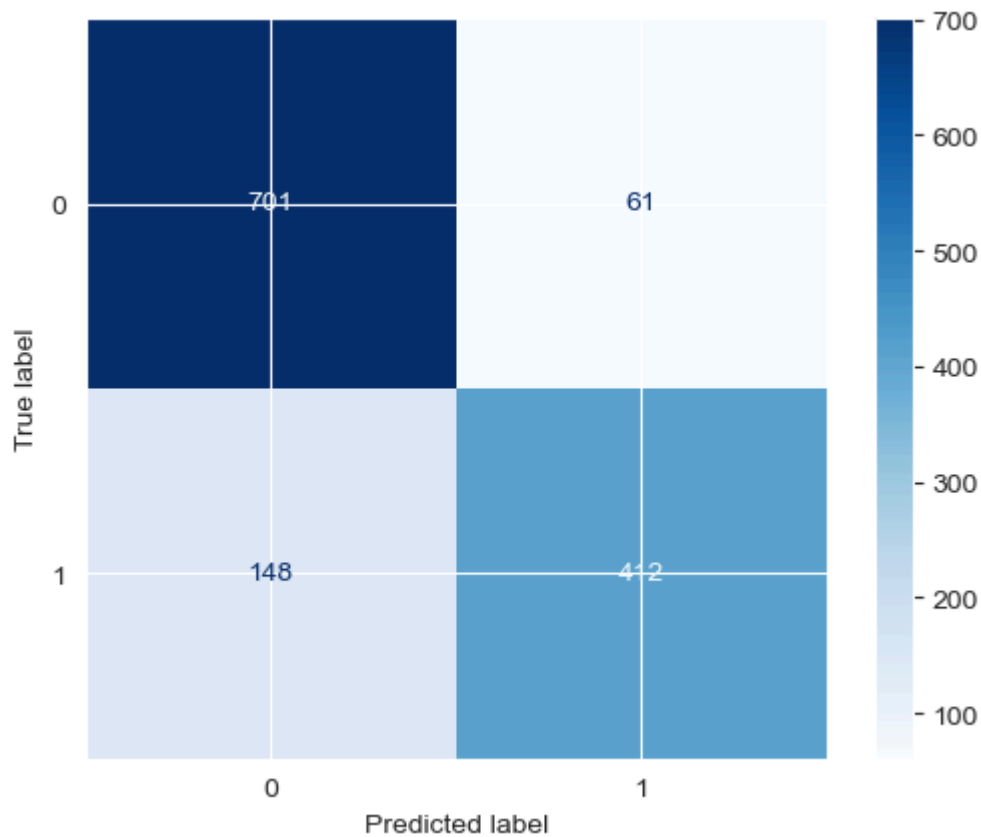
```
Out[12]: 0.8647342995169082
```

```
In [13]: print("Confusion Matrix: \n", metrics.confusion_matrix(y_test.values.ravel(), y_pred))
```

```
Confusion Matrix:
[[686  60]
 [ 80 209]]
```

```
In [42]: mat = ConfusionMatrixDisplay(confusion_matrix=metrics.confusion_matrix(y_test.values.ravel(),
        mat.plot(cmap=plt.cm.Blues)
```

```
plt.show()
```



```
In [43]: print(classification_report(y_test.values.ravel(), y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.92	0.87	762
1	0.87	0.74	0.80	560
accuracy			0.84	1322
macro avg	0.85	0.83	0.83	1322
weighted avg	0.84	0.84	0.84	1322

```
In [44]: print("Accuracy Score: ", accuracy_score(y_test.values.ravel(), y_pred))
```

Accuracy Score: 0.8419062027231468

```
In [45]: print("Precision Score: ", precision_score(y_test, y_pred))
```

Precision Score: 0.8710359408033826

```
In [46]: print("Recall Score: ", recall_score(y_test, y_pred))
```

Recall Score: 0.7357142857142858

```
In [47]: print("F1 Score: ", metrics.f1_score(y_test, y_pred, average="weighted"))
```

F1 Score: 0.8395176432221052

```
In [48]: print("Error :", 1 - accuracy_score(y_test.values.ravel(), y_pred))
```

Error : 0.15809379727685324

Using SVM

```
In [49]: svm_model = SVC()
svm_model.fit(x_train, y_train.values.ravel())
y_pred = svm_model.predict(x_test)
print(metrics.classification_report(y_test.values.ravel(), y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.92	0.87	762
1	0.87	0.74	0.80	560
accuracy			0.84	1322
macro avg	0.85	0.83	0.83	1322
weighted avg	0.84	0.84	0.84	1322

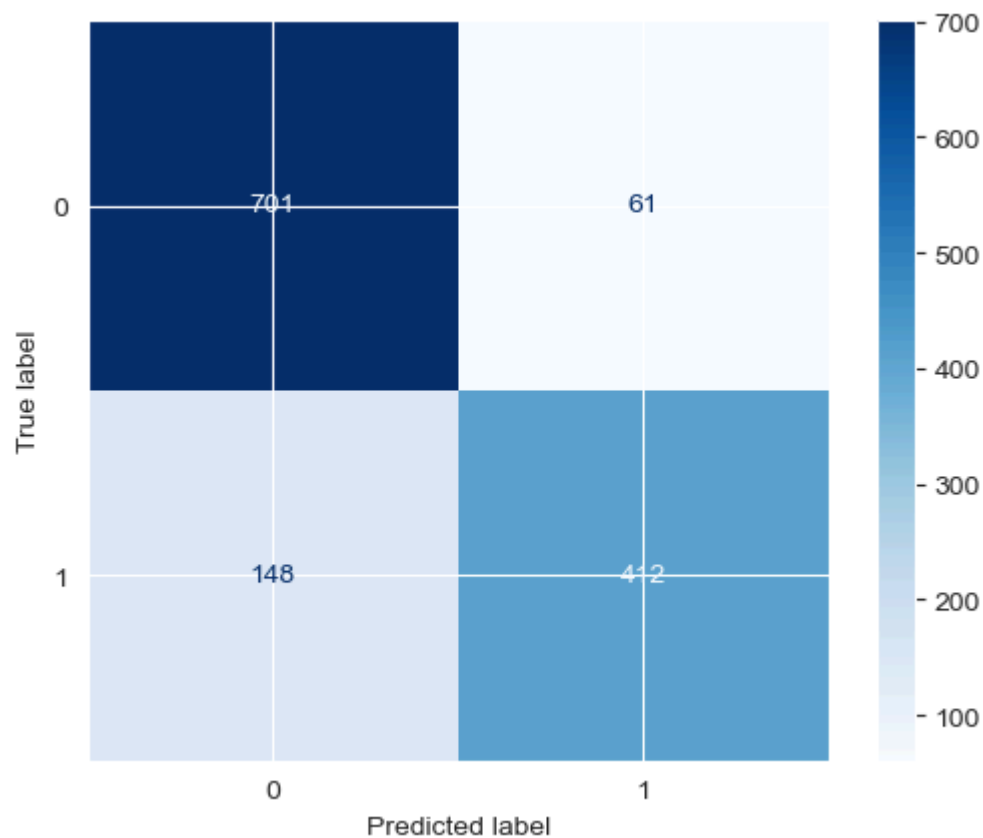
```
In [51]: svm_model.score(x_train,y_train.values.ravel())
svm_model.score(x_test,y_test)
```

Out[51]: 0.8419062027231468

```
In [54]: print("Confusion Matrix : \n ", metrics.confusion_matrix(y_test.values.ravel(), y_pred))
```

Confusion Matrix :
[[701 61]
[148 412]]

```
In [53]: mat = ConfusionMatrixDisplay(confusion_matrix=metrics.confusion_matrix(y_test.values.ravel(),
mat.plot(cmap=plt.cm.Blues)
plt.show())
```



With Upsampling

```
In [32]: spam_data = df[df["Prediction"] == 1]
ham_data = df[df["Prediction"] == 0]
```

```
spam_upsample = resample(spam_data,
                          replace=True,
                          n_samples=int(0.8 * len(ham_data)),
                          random_state=42)
new_df = pd.concat([ham_data, spam_upsample], ignore_index=True)
```

In [33]: `new_df.head()`

```
Out[33]:
```

	the	to	ect	and	for	of	a	you	hou	in	...	connevey	jay	valued	lay	infrastructure	mili
0	0	0	1	0	0	0	2	0	0	0	...	0	0	0	0		0
1	8	13	24	6	6	2	102	1	27	18	...	0	0	0	0		0
2	0	0	1	0	0	0	8	0	0	4	...	0	0	0	0		0
3	0	5	22	0	5	1	51	2	10	1	...	0	0	0	0		0
4	7	6	17	1	5	2	57	0	9	3	...	0	0	0	0		0

5 rows × 3001 columns



In [34]: `new_df.shape`

Out[34]: (6609, 3001)

In [35]: `new_df = new_df.sample(frac=1)`

In [36]: `sns.distplot(new_df["Prediction"])`
`plt.show()`

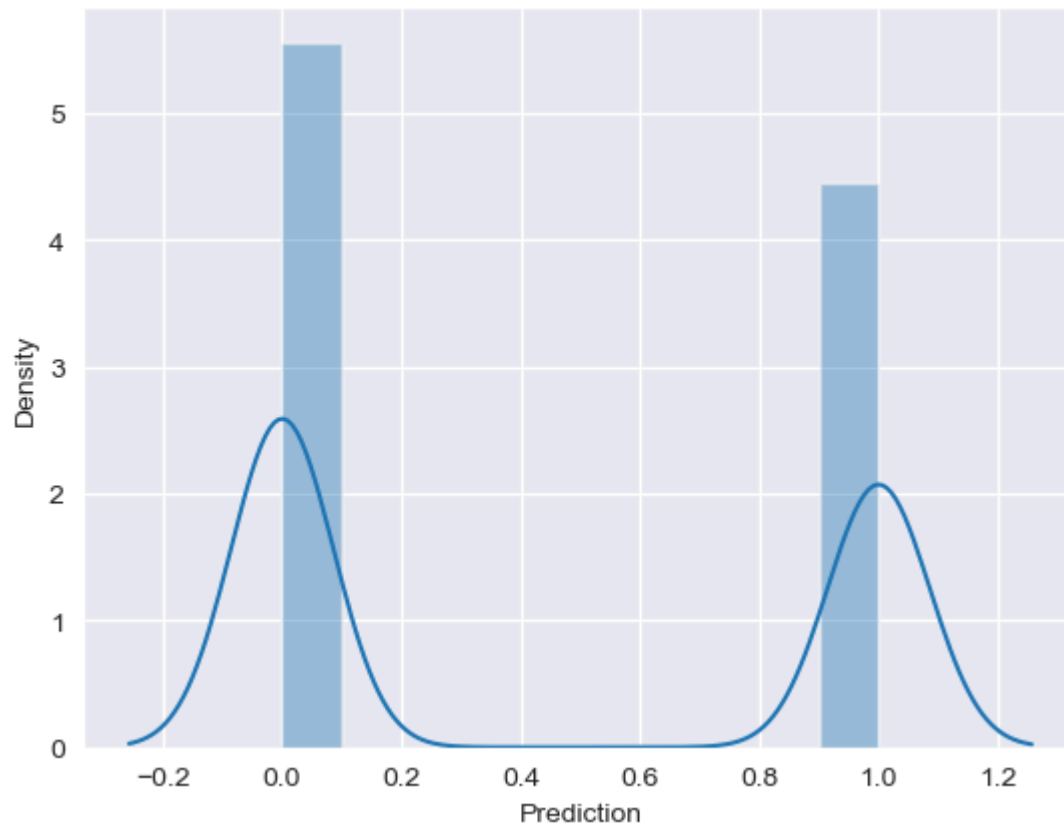
C:\Users\ansha\AppData\Local\Temp\ipykernel_21284\196924750.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(new_df["Prediction"])
```



```
In [37]: x = new_df.drop("Prediction", axis=1)
y = new_df[["Prediction"]]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Using KNN

```
In [38]: knn_model = KNeighborsClassifier(n_neighbors=2)
knn_model.fit(x_train, y_train.values.ravel())
y_pred = knn_model.predict(x_test)
print(metrics.classification_report(y_test.values.ravel(), y_pred))
```

	precision	recall	f1-score	support
0	0.95	0.91	0.93	762
1	0.89	0.94	0.91	560
accuracy			0.92	1322
macro avg	0.92	0.93	0.92	1322
weighted avg	0.92	0.92	0.92	1322

Using SVM

```
In [40]: svm_model = SVC()
svm_model.fit(x_train, y_train.values.ravel())
y_pred = svm_model.predict(x_test)
print(metrics.classification_report(y_test.values.ravel(), y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.92	0.87	762
1	0.87	0.74	0.80	560
accuracy			0.84	1322
macro avg	0.85	0.83	0.83	1322
weighted avg	0.84	0.84	0.84	1322

In []: